

Міністерство освіти і науки України
Національний Університет “Львівська Політехніка”



Лабораторна робота
з дисципліни
“Об’єктно-орієнтоване програмування, частина 2”

Виконав:
студент гр. ІХ-22
Поліщук Юра

Прийняла:
Гордійчук-Бублівська О.В

Львів-2026

Мета: Ознайомитися з принципами SOLID. Навчитися застосовувати їх під час проєктування класів на Python. Формувати навички створення гнучкого, масштабованого та підтримуваного коду.

1. Single Responsibility Principle (SRP)

1.1

Є клас CallReport, який формує звіт про дзвінки та зберігає його у файл. Розділити відповідальності відповідно до SRP.

1.2

Клас Subscriber:

зберігає дані абонента

відправляє SMS

розраховує баланс

Перепроєктувати систему так, щоб кожен клас мав одну відповідальність.

```
py > adada.py > ...
1 class Call:
2     def __init__(self, phone_number: str, duration: int, price_per_min: float):
3         self.phone_number = phone_number
4         self.duration = duration
5         self.price_per_min = price_per_min
6
7
8 class ReportFormatter:
9     def format(self, calls: list[Call]) -> str:
10        report = "CALL REPORT\n"
11        report += "-----\n"
12        total = 0
13
14        for call in calls:
15            cost = call.duration * call.price_per_min
16            total += cost
17            report += f"{call.phone_number} | {call.duration} min | {cost:.2f} UAH\n"
18
19        report += "-----\n"
20        report += f"TOTAL: {total:.2f} UAH\n"
21        return report
22
23 class FileSaver:
24     def save(self, filename: str, content: str):
25         with open(filename, "w", encoding="utf-8") as f:
26             f.write(content)
27
28 class CallReportService:
29     def __init__(self, formatter: ReportFormatter, saver: FileSaver):
30         self.formatter = formatter
31         self.saver = saver
32
33     def generate(self, calls: list[Call], filename: str):
34         report = self.formatter.format(calls)
35         self.saver.save(filename, report)
36
37 class Subscriber:
38     def __init__(self, name: str, phone: str, balance: float):
39         self.name = name
40         self.phone = phone
41         self.balance = balance
42
43 class BalanceCalculator:
44     def charge(self, subscriber: Subscriber, amount: float):
45         subscriber.balance -= amount
46
47     def add_money(self, subscriber: Subscriber, amount: float):
48         subscriber.balance += amount
49
```

```

34         self.saver.save(filename, report)
35
36     class Subscriber:
37         def __init__(self, name: str, phone: str, balance: float):
38             self.name = name
39             self.phone = phone
40             self.balance = balance
41
42
43     class BalanceCalculator:
44         def charge(self, subscriber: Subscriber, amount: float):
45             subscriber.balance -= amount
46
47         def add_money(self, subscriber: Subscriber, amount: float):
48             subscriber.balance += amount
49
50     class SmsSender:
51         def send_sms(self, subscriber: Subscriber, message: str):
52             print(f"SMS to {subscriber.phone}: {message}")
53
54     if __name__ == "__main__":
55
56         calls = [
57             Call("0501112233", 3, 2.5),
58             Call("0675558899", 10, 2.5),
59             Call("0934442211", 1, 2.5)
60         ]
61
62         report_service = CallReportService(ReportFormatter(), FileSaver())
63         report_service.generate(calls, "report.txt")
64         print("Report created: report.txt")
65
66         sub = Subscriber("yura", "0501112233", 50)
67
68         calculator = BalanceCalculator()
69         sms = SmsSender()
70
71         calculator.charge(sub, 10)
72         sms.send_sms(sub, "You were charged 10 UAH")
73
74         calculator.add_money(sub, 20)
75         sms.send_sms(sub, "You received 20 UAH")
76
77         print(f"Balance: {sub.balance} UAH")
78

```

```

[Running] python -u "c:\Users\w10lf\Downloads\hmpyth\py\adada.py"
Report created: report.txt
SMS to 0501112233: You were charged 10 UAH
SMS to 0501112233: You received 20 UAH
Balance: 60 UAH

```

2. Open/Closed Principle (OCP)

2.1

Реалізувати систему тарифікації дзвінків з базовим тарифом.

Додати новий тариф без зміни існуючого коду.

2.2

Система підтримує тарифи: VoiceTariff, DataTariff.

Розширити систему тарифом RoamingTariff, не змінюючи логіку розрахунку вартості

```

ocp_tariffs.py > ocp_tariffs.py > ...
1  from abc import ABC, abstractmethod
2  class Tariff(ABC):
3      @abstractmethod
4      def calculate_cost(self, amount: float) -> float:
5          pass
6  class VoiceTariff(Tariff):
7
8      def calculate_cost(self, minutes: float) -> float:
9          return minutes * 2.0
10
11
12  class DataTariff(Tariff):
13
14      def calculate_cost(self, megabytes: float) -> float:
15          return megabytes * 0.05
16
17  class RoamingTariff(Tariff):
18
19      def calculate_cost(self, minutes: float) -> float:
20          return minutes * 10.0 + 5
21
22  class BillingCalculator:
23      def calculate(self, tariff: Tariff, usage: float) -> float:
24          return tariff.calculate_cost(usage)
25
26  if __name__ == "__main__":
27
28      calculator = BillingCalculator()
29
30      voice = VoiceTariff()
31      data = DataTariff()
32      roaming = RoamingTariff()
33
34      print("Voice call 15 min:", calculator.calculate(voice, 15), "UAH")
35      print("Internet 500 MB:", calculator.calculate(data, 358), "UAH")
36      print("Roaming 3 min:", calculator.calculate(roaming, 2.9), "UAH")

```

```

Voice call 15 min: 30.0 UAH
Internet 500 MB: 25.0 UAH
Roaming 3 min: 35.0 UAH

** Process exited - Return Code: 0 **

```

3. Liskov Substitution Principle (LSP)

3.1

Є базовий клас `NetworkConnection`.

Реалізувати підкласи `LTEConnection`, `WiFiConnection`, які можна взаємозамінно використовувати.

3.2 Виправити створену ієрархію, де клас `SatelliteConnection` порушує очікувану поведінку базового класу (супутник не може працювати як звичайне з'єднання).

```

1  from abc import ABC, abstractmethod
2  > class NetworkConnection(ABC): ...
9
10 class LTEConnection(NetworkConnection):
11     def connect(self):
12         print("LTE: Підключення встановлено через стільникову вежу.")
13
14     def send_data(self, data: str):
15         print(f"LTE: Дані '{data}' надіслано.")
16
17 class WiFiConnection(NetworkConnection):
18 >     def connect(self): ...
20
21     def send_data(self, data: str):
22         print(f"WiFi: Дані '{data}' надіслано.")
23 class SatelliteConnection(ABC):
24 >     """ ...
28     @abstractmethod
29     def establish_satellite_link(self):
30         pass
31
32 class StarlinkConnection(SatelliteConnection, NetworkConnection):
33
34     def establish_satellite_link(self):
35         print("Starlink: Пошук супутників на орбіті...")
36
37     def connect(self):
38         self.establish_satellite_link()
39         print("Starlink: Зв'язок встановлено.")
40
41     def send_data(self, data: str):
42         print(f"Starlink: Дані '{data}' надіслано через супутник.")
43
44
45 def transmit_report(connection: NetworkConnection):
46     print(f"\n--- Робота з {type(connection).__name__} ---")
47     connection.connect()
48     connection.send_data("Звіт за квартал")
49
50 if __name__ == "__main__":
51     connections = [
52         LTEConnection(),
53         WiFiConnection(),
54         StarlinkConnection()
55     ]
56
57     for conn in connections:
58         transmit_report(conn)

```

```

--- Робота з LTEConnection ---
LTE: Підключення встановлено через стільникову вежу.
LTE: Дані 'Звіт за квартал' надіслано.

--- Робота з WiFiConnection ---
WiFi: Підключення встановлено до роутера.
WiFi: Дані 'Звіт за квартал' надіслано.

--- Робота з StarlinkConnection ---
Starlink: Пошук супутників на орбіті...
Starlink: Зв'язок встановлено.
Starlink: Дані 'Звіт за квартал' надіслано через супутник.

** Process exited - Return Code: 0 **

```

Dependency Inversion Principle (DIP)

Система моніторингу мережі напряму використовує FileLogger.

Перепроєктувати систему відповідно до DIP.

Реалізувати можливість підключення:

FileLogger

ServerLogger

ConsoleLogger

(без змін у коді класу NetworkMonitor).

```
py > adada.py > ...
1  from abc import ABC, abstractmethod
2
3  class ILogger(ABC):
4      @abstractmethod
5      def log(self, message: str):
6          pass
7
8  class FileLogger(ILogger):
9      def log(self, message: str):
10         print(f"[File] Запис у файл log.txt: {message}")
11
12  class ServerLogger(ILogger):
13  > def log(self, message: str): ...
14
15
16  class ConsoleLogger(ILogger):
17      def log(self, message: str):
18         print(f"[Console] Виведення: {message}")
19
20  class NetworkMonitor:
21      """
22      Клас не знає про існування File або ServerLogger.
23      Він працює лише з інтерфейсом ILogger.
24      """
25      def __init__(self, logger: ILogger):
26         self.logger = logger
27
28      def check_connection(self):
29
30         status = "Мережа працює стабільно"
31         self.logger.log(status)
32
33  if __name__ == "__main__":
34      print("=== Демонстрація DIP ===\n")
35      monitor_console = NetworkMonitor(ConsoleLogger())
36      monitor_console.check_connection()
37      monitor_file = NetworkMonitor(FileLogger())
38      monitor_file.check_connection()
39      monitor_server = NetworkMonitor(ServerLogger())
40      monitor_server.check_connection()
```

```
[Console] Виведення: Мережа працює стабільно
[File] Запис у файл log.txt: Мережа працює стабільно
[Remote Server] Надсилання логів на сервер: Мережа працює стабільно
```

На даній лабораторній роботі я ознайомився з принципами SOLID.

Навчився застосовувати їх під час проєктування класів на Python.

Формувати навички створення гнучкого, масштабованого та підтримуваного коду